

Unity 背包系统模块拆解案

作者：费锦意

该系统模块应用于本人原创游戏《晦境孤剑：残玉为王》

Unity 2D/3D · C# + ScriptableObject · 事件驱动架构

一、整体架构概览

系统采用 分层架构 + 事件驱动 设计，各层职责清晰、耦合度低：



核心设计理念：

- 数据与视图分离 (Data/View Separation)
- 通过 EventCenter 事件总线解耦各模块，避免直接引用

- 效果系统使用策略模式（Strategy Pattern），扩展零修改成本
- 道具配置数据使用 ScriptableObject，支持编辑器可视化配置

二、数据层：ItemData & BagData

2.1 ItemData（道具配置数据）

类型：ScriptableObject

ItemData 作为道具的数据资产（Data Asset），存储道具的全部静态配置信息，与运行时逻辑完全解耦。

字段	类型	说明
ItemName	string	道具名称，同时作为存档序列化的唯一标识键
ItemIcon	Sprite	道具图标，用于 UI 渲染
ItemDescription	string	道具描述文本（[TextArea] 多行）
ItemLevel	int (1~3)	道具等级，控制 UI 分区显示逻辑
BagLimit	int	背包持有上限，硬性约束
Effects	ItemEffect[]	效果列表，支持多效果叠加（策略模式）

设计要点：

- 使用 [Range(1,3)] Attribute 在 Inspector 限制等级合法范围，防止配置错误
- Effects 数组支持多个 ItemEffect 实例，使道具可同时触发治疗+增益等复合效果
- ScriptableObject 实例在 BagSystem.AllItemData 中统一注册，用于存档反序列化时按名称查找引用

2.2 BagData（背包数据模型）

类型：普通 C# 类（非 MonoBehaviour）

BagData 是纯数据模型，使用 Dictionary<ItemData, int> 存储道具引用与数量的映射关系。

核心接口：

```
bool AddItem(ItemData itemData, int count = 1)    // 添加道具，超上限返回
false
bool RemoveItem(ItemData itemData, int count = 1) // 移除道具，数量不足返回
false
int  GetCount(ItemData itemData)                  // 查询数量，不存在则返回
0
Dictionary<ItemData, int> GetAllItems()           // 获取全部道具
List<KeyValuePair<ItemData, int>> GetItemsByLevel(int level) // 按等级筛
选
void Clear()                                       // 清空背包
```

实现细节：

- `AddItem`：通过 `GetValueOrDefault` 安全读取当前数量，与 `BagLimit` 比较后再写入，原子操作避免数量越界
- `RemoveItem`：数量归零时调用 `_items.Remove(itemData)` 清除字典条目，避免残留无效 Key
- `GetItemsByLevel`：使用 LINQ `Where` 按 `ItemLevel` 过滤，返回 `List<KeyValuePair<ItemData, int>>`，供 UI 层按分区渲染

三、业务层： `BagSystem`

类型： `MonoBehaviour` + 单例模式（Singleton Pattern）

`BagSystem` 是背包系统的唯一入口，对外暴露增删查用接口，对内将数据操作委托给 `BagData`，同时负责事件广播与持久化存档。

3.1 单例初始化

```
private void Awake()
{
    if (Instance != null && Instance != this) { Destroy(gameObject);
return; }
    Instance = this;
    BagData = new BagData();
}
```

采用**标准单例守卫**模式，确保场景中只有一个实例，重复加载时自动销毁多余实例。

3.2 核心业务接口

AddItem — 拾取道具

```
调用 BagData.AddItem → 失败则 Log 并 return false
成功 → 广播 ItemPickUp 事件（携带 ItemData 参数）
        广播 BagChanged 事件（通知 UI 刷新）
```

RemoveItem — 移除道具

```
调用 BagData.RemoveItem → 失败则 Log 并 return false
成功 → 广播 BagChanged 事件
```

UseItem — 使用道具

```
检查数量 > 0 → 调用 BagData.RemoveItem(1)
                → 调用 ApplyEffects（遍历 Effects 数组逐一执行）
                → 广播 ItemUse 事件
                → 广播 BagChanged 事件
```

ApplyEffects — 效果施加

```
foreach (var effect in itemData.Effects)
    effect.Apply(target);
```

遍历效果列表，调用多态 `Apply` 方法，完全依赖抽象接口，无需感知具体效果类型（开闭原则）。

3.3 持久化存档

序列化方案： `JsonUtility + File.WriteAllText`，存储路径为 `Application.persistentDataPath/bag_save.json`

Save 流程：

1. 遍历 `BagData.GetAllItems()`
2. 将 `Dictionary<ItemData, int>` 转换为可序列化的 `BagSaveData`（包含 `BagEntry[]`，字段为道具名 + 数量）
3. `JsonUtility.ToJson` 序列化后写入磁盘

Load 流程：

1. 读取 JSON 文件， `JsonUtility.FromJson` 反序列化
2. 遍历 `entries`，调用 `FindItemByName` 按名称在 `AllItemData` 中查找对应 `ScriptableObject` 引用
3. 调用 `BagData.AddItem` 还原背包状态
4. 广播 `BagChanged` 通知 UI 刷新

注意事项：

- `ScriptableObject` 无法直接序列化，因此存档存储 `ItemName` 字符串作为间接引用键
 - `AllItemData` 数组需在 Inspector 中手动绑定所有可能出现的道具资产，确保反序列化时能正确查找
 - `OnApplicationQuit` 触发自动保存，防止数据丢失
-

四、效果系统： `ItemEffect` / `HealEffect` / `TimedBoostEffect`

4.1 抽象基类 `ItemEffect`

```
public abstract class ItemEffect : ScriptableObject
{
    public abstract void Apply(GameObject target);
    public abstract void Remove(GameObject target);
}
```

使用**策略模式**，将道具效果抽象为独立的 ScriptableObject 子类，新增效果类型无需修改任何现有代码，只需：

1. 继承 ItemEffect，实现 Apply / Remove
2. 在 Inspector 中将新效果资产拖入 ItemData.Effects 数组

符合开闭原则 (Open/Closed Principle)： 对扩展开放，对修改关闭。

4.2 HealEffect（即时治疗效果）

- 使用 [CreateAssetMenu] 支持在编辑器右键创建资产
- Apply：获取目标的 PlayerStat 组件，使用 Mathf.Min 回血并限制不超过 HP 上限
- Remove：即时效果无需逆操作，空实现

4.3 TimedBoostEffect（定时增益效果）

核心设计：协程载体组件模式

ScriptableObject 本身无法直接运行协程（非 MonoBehaviour），通过在运行时动态挂载私有内部类 TimedBoostRunner : MonoBehaviour 作为协程载体来驱动定时逻辑：

```
Apply() 被调用
→ 修改 PlayerStat 属性（乘以 BoostValue）
→ target.AddComponent<TimedBoostRunner>()
→ runner.StartCoroutine(BoostCoroutine)
    └─ WaitForSeconds(Duration)
    └─ Remove()：属性除以 BoostValue 复原
    └─ Destroy(runner)：清理载体组件
```

字段	类型	说明
----	----	----

BoostType	enum	增益类型（SpeedBoost / DamageBoost）
BoostValue	float	增益倍率，乘法计算
Duration	float	持续时间（秒）

潜在改进点：

- 当前实现为直接修改基础属性，多个同类增益叠加时除法恢复可能导致数值漂移；可改为**属性修改器（Modifier）系统**统一管理
- DamageBoost 分支目前仅打 Log，未实际修改属性，需补全

五、UI 层：BagUI & BagSlot

5.1 BagUI（背包面板控制器）

事件驱动的生命周期管理：

```
EventCenter
├─ OpenBag   → OnOpenBag()   : SetActive(true)  + Refresh()
├─ CloseBag  → OnCloseBag()  : SetActive(false) + HideTooltip()
└─ BagChanged → Refresh()    : 重新生成所有格子
```

刷新策略：全量销毁重建

```
Refresh()
├─ ClearSlots() : Destroy 所有 _slotInstances, 清空列表
├─ CreateSlotsForLevel(1, _level1Container)
├─ CreateSlotsForLevel(2, _level2Container)
└─ CreateSlotsForLevel(3, _level3Container)
```

每次刷新先销毁所有格子实例再重建，简单可靠，适合背包容量较小的场景。若背包容量大、刷新频繁，可优化为**对象池（Object Pool）+ Diff 更新策略**。

格子实例化：

- _slotTemplate 预制体在 Start 中 SetActive(false) 隐藏作为模板
- 实例化时 Instantiate(_slotTemplate, container)，然后 SetActive(true)

- 实例引用存入 `_slotInstances` 列表，确保后续清理不遗漏

5.2 BagSlot（单个道具格子）

实现三个 Unity UI 事件接口，处理用户交互：

接口	触发时机	行为
<code>IPointerClickHandler</code>	左键点击	调用 <code>_bagUI.UseItem(_itemData)</code>
<code>IPointerEnterHandler</code>	鼠标移入	调用 <code>_bagUI.ShowTooltip(_itemData)</code>
<code>IPointerExitHandler</code>	鼠标移出	调用 <code>_bagUI.HideTooltip()</code>

职责隔离：BagSlot 不持有 BagSystem 引用，所有业务操作均通过 `_bagUI` 回调，符合最少知识原则（Law of Demeter）。

六、场景层：ItemPickup

功能概览

- 上下浮动动画（正弦波位移）
- 触发器检测玩家进入/离开范围，广播 ShowTips / HideTips 事件
- F 键：拾取入背包（受 BagLimit 限制）
- E 键：直接使用，不入背包（绕过库存上限）

浮动动画

```
transform.position = _originPos + Vector3.up * Mathf.Sin(Time.time * BobSpeed) * BobAmplitude;
```

以 `_originPos`（Start 时记录的初始位置）为基准，每帧叠加正弦偏移，参数 `BobAmplitude`（幅度）和 `BobSpeed`（频率）可在 Inspector 调节。

输入处理

按键	行为	说明
----	----	----

F	BagSystem.AddItem → 成功则 Destroy(gameObject)	拾取入背包，满则无法拾取，道具保留
E	BagSystem.ApplyEffects → Destroy(gameObject)	直接使用，不受背包上限限制

七、事件总线：EventCenter 通信映射

事件名	发布方	订阅方	携带参数
OpenBag	BagUI（键盘输入）	BagUI	无
CloseBag	BagUI（键盘输入）	BagUI	无
BagChanged	BagSystem	BagUI	无
ItemPickUp	BagSystem.AddItem	其他系统	ItemData
ItemUse	BagSystem.UseItem / ItemPickup	其他系统	ItemData
ShowTips	ItemPickup	TipsUI 等	ItemData
HideTips	ItemPickup	TipsUI 等	无
SaveGame	外部存档系统	BagSystem	无
LoadGame	外部存档系统	BagSystem	无

八、设计模式总结

模式	应用位置	作用
单例模式	BagSystem	全局唯一访问点，管理背包核心逻辑
策略模式	ItemEffect 体系	效果可任意组合，新增效果无需改动现有代码
观察者模式	EventCenter	解耦模块间通信，支持一对多广播
模板方法模式	ItemEffect 抽象类	定义 Apply/Remove 契约，子类实现具体行为

数据对象模式	ItemData ScriptableObject	配置数据资产化，与运行时逻辑解耦
组合模式	ItemData.Effects 数组	多效果自由组合，复合道具零额外代码

感谢您的浏览！